Objets, Interfaces et polymorphisme

Jean-Luc Falcone

2 mars 2021

Classes dérivées

Dérivation de classe (1)

En Java toutes les classes dérivent d'une autre classe (sauf une).

Par défaut, toutes les classes dérivent de la class `java.lang.Object`.

La classe `Object` ne dérive d'aucune autre classe.

Dérivation de classe (2)

En java, la dérivation a deux effets simultanés, mais distincts:

1. Le sous-typage
2. L'héritage

Ces deux concepts sont souvent confondus.

Sous-typage

Un type T est un sous-type d'un type U si et seulement si toutes les valeurs de T sont aussi des valeurs de U. Notation:

$$T <: U \quad \Longleftrightarrow \quad \forall t,\ t : T \rightarrow t : U$$

Si $T <: U$, alors on peut assigner une valeur du type T à une référence du type U:

```
T t = new T();
U u = t;        //Compile seulement si T<:U
```

Sous-typage: transitivité

La relation de sous-typage est transitive:

$$T <: U \text{ et } S <: T \implies S <: U$$

Type à la compilation vs. exécution (1)

Comme toutes les classes en Java sont des sous-type de `Object`, on peut toujours assigner une référence de type `Object`.

Le compilateur perd la trace du type sous-jacent.

```
Point p = new Point( 2.5, 1.2 );
Point q = new Point( 15, -1.0 );


Object o = p;
o.distance(q); //Erreur compilation
q.distance(o); //Erreur compilation
```

Type à la compilation vs. exécution (2)

On peut tester le type sous-jacent d'une instance à l'exécution avec l'opérateur `instanceof`.

```
Object o1 = new Point(2.3, 1.2);
Object o2 = "Hello, world !";

o1 instanceof Point;  //true
o1 instanceof Object; //true (toujours vrai)
o2 instanceof Point;  //false
o2 instanceof String; //true
```

Type à la compilation vs. exécution (3)

On peut forcer la conversion d'un type à l'exécution. Une erreur d'exécution se produit lorsque les types sont incompatibles:

```
Object o1 = new Point(2.3, 1.2);
Object o2 = "Hello, world !";

Point p = (Point) o1;
Point q = (Point) o2; //Erreur d'exécution
```

Type à la compilation vs. exécution: Remarques

Le *type cast* n'affecte que la référence. Cela n'affecte pas l'intance elle-même.

Pour éviter les surprises, il faudrait le protéger dans un `if`:

```
if( o instanceof Point ) {
  Point p = (Point) o;
} else {
  //Ne pas oublier de gérer ce cas
}
```

Héritage

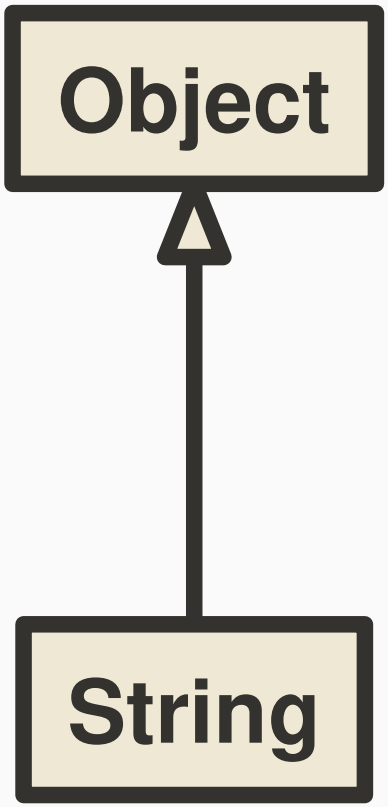
L'héritage est un mécanisme qui permet à une classe d'obtenir la structure et le comportement de la classe dont elle dérive (champs et méthodes).

Ce mecanisme est complexe, souvent mal compris, et très souvent mal utilisé.

En Java toute classe ne peut hériter que d'une seule classe, `Object` par défaut.

Donc, par transitivité toute classe est un sous-type d'`Object`.

Héritage: Représentation UML



En UML on note l'héritage par une flêche à la point blanche.
On dit que la classe `String` spécialise la classe `Object`
Réciproquement la classe `Object` généralise la classe `String`.

Héritage de `Object`

Comme toutes les classes dérivent d'`Object`, toutes les classes ont une implémentation par défaut de plusieurs méthodes. Notamment:

```
public String toString(); //Instance en String

public boolean equals(Object o); //Egalité

public int hasCode(); //Code de hashage
```

On peut cependant redéfinir ces méthodes dans la classe dérivée (*override*).

Identité (==)

L'opérateur `==` retourne vrai si, et seulement si, deux références pointent sur la même instance:

```
Point p = new Point(1,2);
Point q = new Point(1,2);
Object o = p;

p == p; //vrai
p == q; //faux instances différentes
p == o; //vrai
```

Egalité (equals)

Par défaut la méthode **equals** fonctionne comme l'opérateur ==:

```
Point p = new Point(1,2);
Point q = new Point(1,2);
Object o = p;

p.equals(p); //vrai
p.equals(q); //faux instances différentes
p.equals(q); //vrai
```

Egalité: redéfinition (1)

Exemple selon les bonnes pratiques:

```
class Point {
  private final double x;
  private final double y;
  /*...*/
  public boolean equals( Object o ) {
        if( o == this ) return true;
        if( o == null ) return false;
        if( ! o isinstance Point ) return false;
        Point that = (Point) o;
        return that.x == x && that.y == y;
  }
}
```

Egalité: redéfinition (2)

```
Point p = new Point(1,2);
Point q = new Point(1,2);
Object o = p;

p.equals(p); //vrai
p.equals(q); //vrai, cf redéfinition
p.equals(q); //vrai
```

Egalité: redéfinition (3)

En redéfinissant l'égalité, il faut respecter les règles suivantes:

```
//Pour tout x,y et z
x.equals(x) == true;
x.equals(y) == y.equals(x);
if( x.equals(y) && y.equals(z)  ) x.equals(z);
if( x != null ) ! x.equals(null);
x.equals(y) == x.equals(y);
```

Code de hachage

Le code de hachage (*hash code*) est une valeur numérique calculée pour chaque instance.

Utilisée dans la librairie standard (par exemple `HashMap`).

Règle: Si `x.equals(y)` alors `x.hashCode()` est égal à `y.hashCode()`.

La réciproque n'est pas nécéssaire.

Corrolaire: Si on redéfinit `equals`, il faut redéfinit `hashCode`.

Redéfinition: `hashCode` (1)

Critères à respecter (en plus de la règle), pour la fonction `hashCode`:

- aussi rapide que possible
- se base sur les mêmes champs qu'`equals`
- consistente d'un appel à l'autre
- retourne des valeurs bien réparties sur l'espaces des nombres entiers (`int`).

Redéfinition: `hashCode` (2)

Voici quelques pistes pour redéfinir un `hashCode`:

- Pour les membres qui sont des objets, utiliser la fonctions statiques `Objects.hashCode()`
- Pour les nombres entiers utiliser leur valeur telle quelle.
- Pour les nombre à virgule flottante, utiliser les fonctions statiques `Double.hashCode()` ou `Float.hashCode()`.
- Combiner les résultats obtenus en multipliant par des nombres premiers et en additionnant (arbitraires).

Redéfinition: hashCode (2)

```
class Point {
  private final double x;
  private final double y;
  /*...*/
  public int hashCode() {
        int h = 17;
        h = h*31 + Double.hashCode(x);
        h = h*31 + Double.hashCode(y);
        return h;
  }
}
```

Value Object vs. Entity

Deux types principaux d'objets:

1. Les objets-valeurs (*value objects*)
2. Les entités (*entity*)

Tiré de *Domain Driven Design*, E. Evans (2004)

Value Objects

Définis uniquement par la valeur de leurs champs.

Si les valeurs sont les mêmes, on considère que c'est le même objet.

Exemples:

- Couleurs
- Dates
- URL
- Point dans l'espace

Entité

N'est pas déinis par la valeur de ses champs.

Un même objet peut avoir des valeurs différentes au cours du temps:

Exemples:

- Personne
- Reservation
- Compte en banque

Recommandations

	Value object	*Entity*
Immutabilité	toujours	souvent
Redéfinir `equals`	toujours	à éviter
Identifiant unique	inutile	encouragé
Gérer le cycle de vie	inutile	nécéssaire

Interface

Polymorphisme

En informatique, on appelle polymorphisme le fait d'avoir plusieurs comportement rattaché à seul symbol.

On verra ce semestre trois types de polymorphisme:

1. Polymorphisme ad-hoc
2. Polymorphisme par sous-typage
3. Polymorphisme par paramétricité

Polymorphisme ad-hoc: Surcharge de méthode

```
/* Lit sur le système de fichier */
static Image load( File file ) { /*...*/ }

/* Lit sur le réseau */
static Image load( URL url ) { /*...*/ }
```

Polymorphisme ad-hoc: Surcharge d'opérateurs

```
/* addition d'entiers */
2 + 1;

/* concaténation de strings */
"deux" + "un";
```

Polymorphisme par sous-typage

Différents sous-types d'un même type peuvent avoir différentes implémentations pour les mêmes méthodes.

Deux manières de faire en Java:

- Héritage de classe
- Implémentation d'Interface

Interfaces

Abstraction permettant d'exprimer des contraintes sur la présence de méthodes et leur signature.

Définit une forme de contrat, analogue à un protocole de communication.

Mécanisme de sous-typage.

Notification d'alertes (1)

But: intégrer un système d'envoi d'alertes en cas de problèmes.

- Deux niveaux d'alerte: `WARNING` et `CRITICAL`.
- Doit contenir la date et l'heure.
- Plusieurs sorties envisagés:
  - Ecrire sur la console (`System.err.println`).
  - Sauver dans un fichier de log.
  - Envoyer un mail.
  - Utiliser l'API de *Signal* (ou *Slack*, ou un bot *Discord*)
  - Combinaison des options ci-dessus

Notification d'alertes (2a)

```java
import java.time.LocalDateTime;
public class Alert {
  private void format(String level, String msg) {
        var t = LocalDateTime.now();
        return t + " " + level + " " + msg;
  }
  public void warning(String msg) {
        System.err.println(
          format("WARNING", msg));
  }
  public void critical(String msg) {
        System.err.println(
          format("CRITICAL", msg));
  }
}
```

Notification d'alertes (2b)

```
import java.io.File;
public void checkFiles(File[] files, Alert alert){
  for( int i=0; i<files.length; i++ ) {
        final var f = files[i];
        if( !f.exists() )
          alert.critical("File "
                 + f + " missing.");
        else if( ! f.canRead() )
          alert.warning("File " + f
                 + " not readable.");
  }
}
```

Notification d'alertes (3a)

```
import my.library.MailService;
public class Alert {

  private final String[] warnAddress;
  private final String[] critAddress;
  private final MailService mailer;

  public Alert( MailService mailer;
          String[] warnAddress,
          String[] critAddress ) {
          this.mailer = mailer;
          this.warnAddress = warnAddress;
          this.critAddress = critAddress;
  }
  /*...*/
```

Notification d'alertes (3b)

```
public class Alert {
  /* ... */
  public void warning(String msg) {
        for( int i=0; i<warnAddress.size; i++ ) {
          mailer.send( warnAddress[i], //dest
                                "WARNING", //subjet
                                msg );     //body
        }
  }
  /*...*/
}
```

Notification d'alertes (3c)

```
import java.io.File;
public void checkFiles(File[] files, Alert alert){
  for( int i=0; i<files.length; i++ ) {
        final var f = files[i];
        if( !f.exists() )
          alert.critical("File "
                 + f + " missing.");
        else if( ! f.canRead() )
          alert.warning("File " + f
                 + " not readable.");
  }
}
```

Notification d'alerte: interface

Définition des méthodes commune dans une interface:

```
public interface Alert {
  //Pas d'implémentation,
  //  les méthodes sont abstraites
  public void warning( String msg );
  public void critical( String msg );
}
```

Les interfaces

Syntaxe de la déclaration similaire à celles d'une classe:

- Méthodes abstraites (non implémentées)
- Pas de champs
- Pas de membres privés
- Fonctions et constantes statiques possibles
- Depuis JSE8, implémentations par défaut.

Les intefaces: implémentation

Classe peut implémenter une ou plusieurs interfaces:

- Mot clé: `implements`
- Doit implémenter toutes les méthodes abstraites

Introduit une relation de sous typage.

```
/* String <: CharSequence */
class String implements CharSequence {
  /*...*/
}
```

Notification d'alerte: implémentation (1)

```java
import java.time.LocalDateTime;
public class ConsoleAlert implements Alert {
  private void format(String level, String msg) {
        var t = LocalDateTime.now();
        return t + " " + level + " " + msg;
  }
  public void warning(String msg) {
        System.err.println(
          format("WARNING", msg));
  }
  public void critical(String msg) {
        System.err.println(
          format("CRITICAL", msg));
  }
}
```

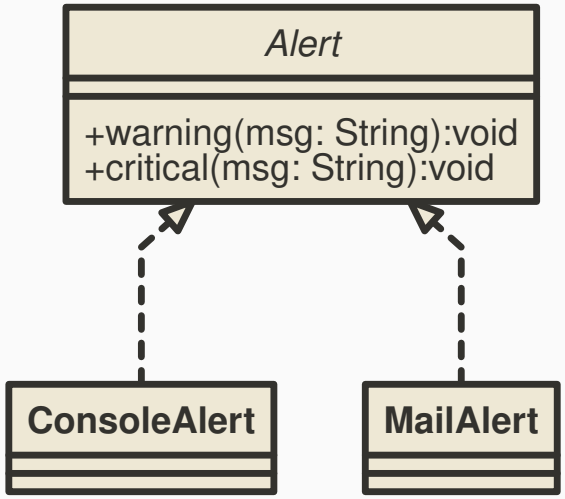
Notification d'alerte: implémentation (2)

```
import my.library.MailService;
public class MailAlert implements Alert {

  private final String[] warnAddress;
  private final String[] critAddress;
  private final MailService mailer;

  public Alert( MailService mailer;
          String[] warnAddress,
          String[] critAddress ) {
          this.mailer = mailer;
          this.warnAddress = warnAddress;
          this.critAddress = critAddress;
  }
  /*...*/
```

Notification d'alerte: utilisation

```
import java.io.File;
public void checkFiles(File[] files, Alert alert){
  for( int i=0; i<files.length; i++ ) {
        final var f = files[i];
        if( !f.exists() )
          alert.critical("File "
                 + f + " missing.");
        else if( ! f.canRead() )
          alert.warning("File " + f
                 + " not readable.");
  }
}
```

Interface: Représentation UML



On UML on dénote une interface par l'utilisation d'italique dans le nom.

L'implémentation est représentée par une flêche à la pointe blanche, en traitillé.

En jargon UML on dit `ConsoleAlert` réalise l'interface `Alert`.

Notification d'alertes: alertes multiples (1)

```
public class MultiAlert implements Alert {
  private final Alert[] warnAlerts;
  private final Alert[] critAlerts;
  public MultiAlert(
        Alert[] warnAlerts,
        Alert[] critAlerts
  ) {
         this.warnAlerts = warnAlerts;
         this.critAlerts = critAlerts;
  }
  /*...*/
}
```

Notification d'alertes: alertes multiples (2)

```
public class MultiAlert implements Alert {
  /*...*/
  public void warning( String msg ) {
        for( int i=0; i<warnAlerts; i++ ) {
          warnAlerts[i].warning(msg);
  }
  public void critical( String msg ) {
        for( int i=0; i<critAlerts; i++ ) {
          crittAlerts[i].critical(msg);
  }
}
```

Notification d'alertes: alertes multiples (3)

Envoyer les *warnings* sur la console, et les *critical* sur la console et par email.

```
var console = new ConsoleAlert();
var email = new MailAlert(ms,
  new String[0],
  new String[] {"alice@truc.com","bob@machin.com"}
);
Alert a = new MultiAlert(
   new Alert[] {console},
   new Alert[] {console,email}
);
```

Fake it till you make it!

Fake it till you make it ! (1)

On peut utiliser des interface pour remplacer un composant qui n'est pas encore écrit.

On peut produire un semblant d'implémentation (en anglais *mock implementation* ou *dummy object*), pour les tests.

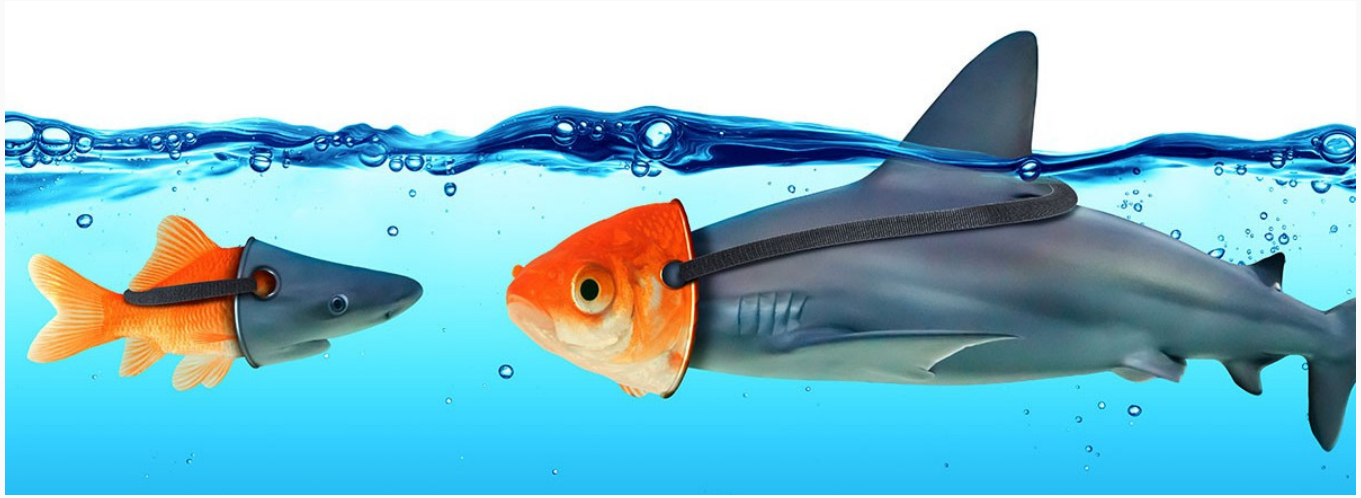


Image tirée du site *ABA For Law Student* (2017).

Fake it till you make it! (2)

```
public interface MailService {
  public void send( String recipient,
        String subject, String body );
}

public interface Authenticator {
  public boolean check( String userid,
        String password );
}
```

Fake it till you make it! (3)

```
public class TestMailer implements MailService {
  private String lastRecipient = "";
  private String lastSubject = "";
  private String lastBody = "";
  /* ... field getters ommitted */
  public void send( String recipient,
        String subject, String body ) {
        last.recipient = recipient;
        last.subject = subject;
        last.body = body;
  }
}
```

Fake it till you make it! (4)

```
public class FakeAuth implements Authenticator {
  private final String password;
  public FakeAuth( pw ) { password = pw; }
  public boolean check( String userid,
        String password ) {
          return this.password.equals( password );
  }
}
```

Quelques remarques

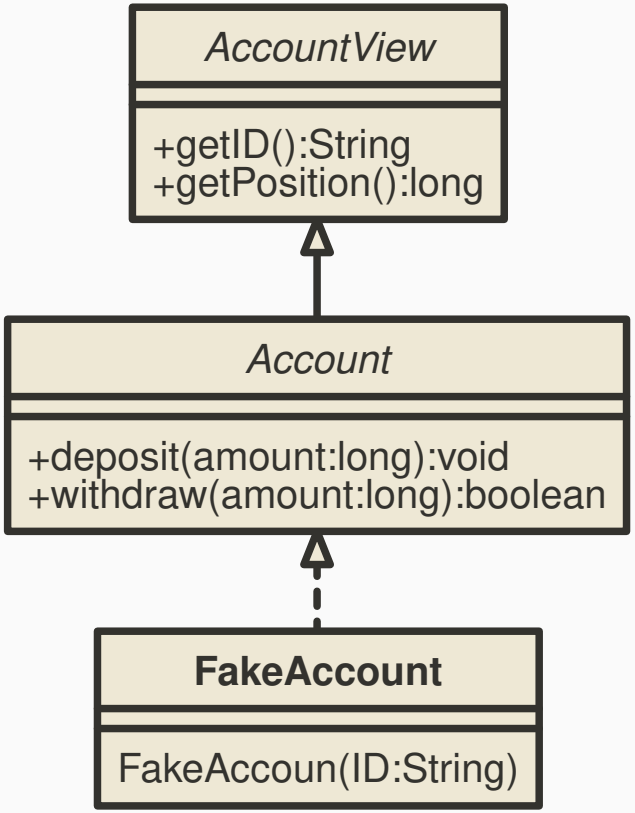
Extension d'interfaces (1)

```
public interface AccountView {
  public String getID();
  public long getPosition();
}


public interface Account
  extends AccountView {
  public void deposit(long amount);
  public boolean widthdraw(long amount);
}
```

Extension d'interfaces (2)

```
class FakeAccount extends Account {
  private long amount = 0;
  private final String ID;
  FakeAccount(ID String) { this.ID=ID; }
  public String getID() { return ID; }
  public long getPosition() { return amount; }
  public void deposit(long amount) {
        this.amount += amount; }
  public boolean widthdraw(long amount) {
        if( this.amount >= amount ) {
          this.amount -= amount;
          return true;
        } else return false;
  }
}
```

Interface: Représentation UML



On a les relations de sous-typage suivantes:

- `Account <: AccountView`
- `FakeAccount <: Account`
- `FakeAccount <: AccountView`

Extension d'interfaces (3a)

```
void prettyPrint( AccountView acc ) {
  System.out.println(
        acc.getID() + "=" + acc.getPosition() );
}

AccountView accountByID( String ID ) {...};

prettyPrint( accountByID("123-456") );
prettyPrint( accountByID("123-456") );
```

Extension d'interfaces (3b)

```
boolean transfer( Account from, Account to,
  long amount ) {
  if( from.withdraw(amount) ) {
        to.deposit(amount);
        return true;
  } else return false;
}

//Ne compile pas
prettyPrint( accountByID("123-456"),
  accountByID("123-456") );
```

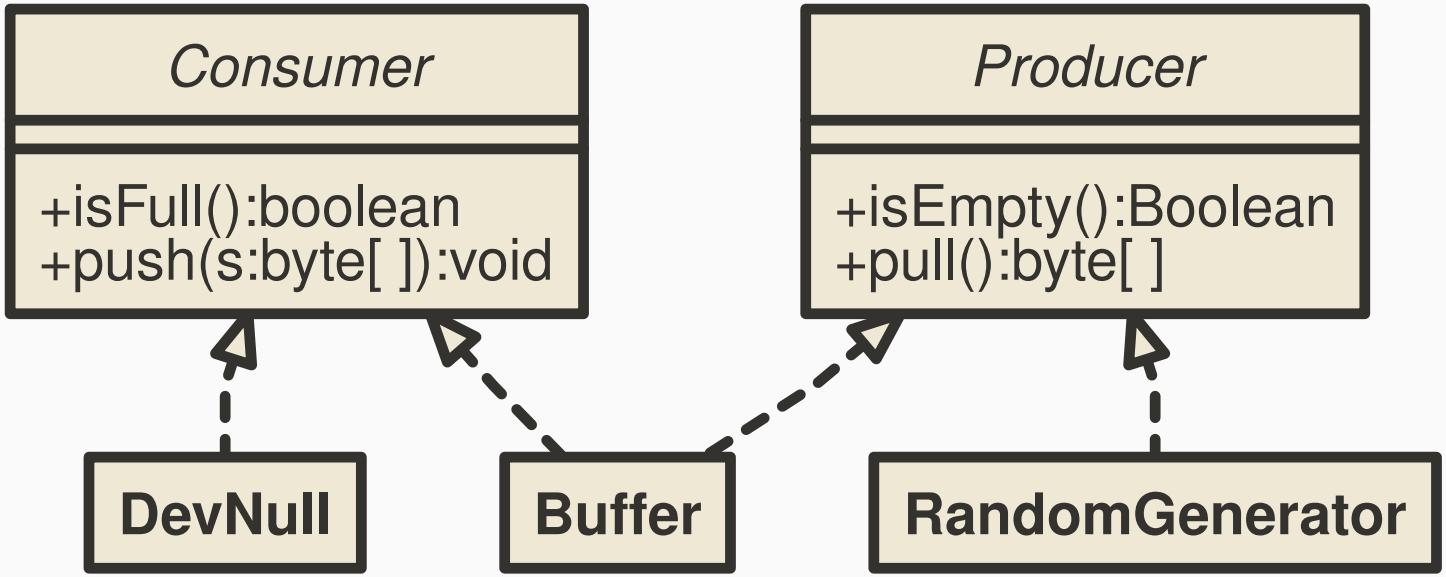
Types plutôt que des commentaire

```
//Ne modifie pas 'acc'
void prettyPrint( AccountView acc ) {
  //...
}

//Peut modifier 'from' et 'to'
boolean transfer( Account from, Account to,
        long amount ) { /*...*/ }

//Peut modifier 'x' mais pas 'y'
long f( Account x, AccountView y ) { //...
```

Attention aux petits malins (et aux idiots)

```
void prettyPrint( AccountView acc ) {
  if( acc instanceof Account ) {
        ( (Account) acc ).deposit(1_000_000);
  }
  System.out.println(
        acc.getID() + "=" + acc.getPosition() );
}
```

Autre exemple: Poducteur-Consommateur



Limitations des interfaces

Pas de constructeurs abstraits:

- Remplacé par les *abstract factory* plus tard dans le cours.

Pas de champs abstraits:

- Raison supplémentaire pour garder tous les champs privés.

Polymorphisme hors de la POO

Systèmes POSIX, en C:

- opérations sur les descripteurs de fichiers
- gestion des adresses pour les sockets: IPv4, IPv6, Unix Sockets

```
struct sockaddr_in server; //Adresse IPv4
/* ... */

//'struct sockaddr' est la "structure parent"
bind(fd, (struct sockaddr *) &server,
  sizeof(server));
```

Attention aux excès

L'abus de polymorphisme et d'encapsulation peut rendre le code:

- Lent
- Gourmand en mémoire
- Illisible
- Indébuggable